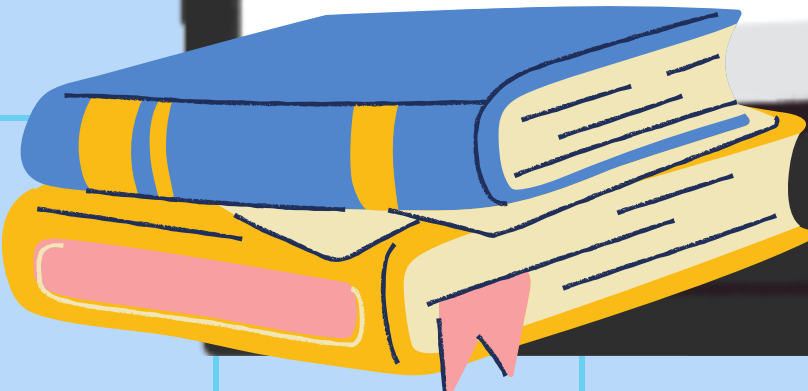
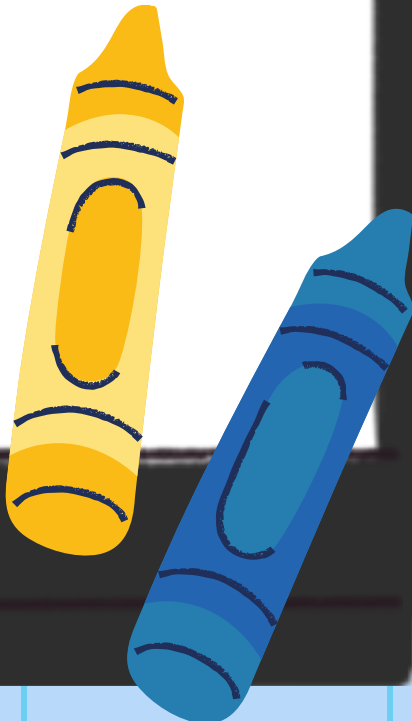


PROJET DOCKER : GESTION DE FLUX DE DONNEES "GESTION DES ETUDIANTS INSCRITS"

Présentation par:
HABBOUBI Mariem & MEZLINI Bochra



INTRODUCTION

- Démonstration d'une architecture conteneurisée modulaire et scalable.
 - Utilisation de Docker pour la gestion des flux de données.
 - Services interconnectés pour la collecte, le traitement et l'affichage.
- 
- 

OBJECTIFS DU PROJET

- Portabilité : Création d'une architecture transportable via Docker.
- Interopérabilité : Communication fluide entre services grâce aux volumes partagés.
- Modularité : Services indépendants pour un développement agile.
- Scalabilité : Capacité à gérer des charges élevées via un répartiteur de charge.

STRUCTURE DE L'ARCHITECTURE

Services Clés :

1. Service de Remplissage : Collecte de données brutes.
2. Service de Lecture : Traitement et enrichissement des données.
3. Service d’Affichage : Interface web pour visualiser les données.
4. Load Balancer : Optimisation de la répartition des requêtes.



FONCTIONNEMENT DES SERVICES



1. SERVICE DE REMPLISSAGE

- Rôle : Génère des données brutes (exemple : données simulées ou provenant de capteurs).
- Technologies : Script Python utilisant des bibliothèques comme random et os pour générer des datasets simples.
- Échange de données : Écrit les données dans un fichier JSON stocké dans un volume partagé Docker.
- Interaction : Partage les données avec le service de lecture via le volume commun shared-data.

```
remplissage > script.py > add_sample_data
1 import json
2 import os
3 import time
4
5 DATA_FILE = "/data/processed_table.json"
6
7 def read_data():
8     """Lit les données du fichier JSON, s'il existe."""
9     if os.path.exists(DATA_FILE):
10         with open(DATA_FILE, 'r') as f:
11             return json.load(f)
12     return []
13
14 def write_data(new_entries):
15     """Écrit de nouvelles données dans le fichier JSON."""
16     data = read_data()
17     data.extend(new_entries)
18     with open(DATA_FILE, 'w') as f:
19         json.dump(data, f, indent=4)
20
21 def add_sample_data():
22     """Ajoute des entrées exemple."""
23     new_entries = [
24         {"Nom": "Alice", "Age": 30, "Ville": "Paris"},
25         {"Nom": "Bob", "Age": 25, "Ville": "Lyon"}
26     ]
27     print("Ajout de nouvelles données :", new_entries)
28     write_data(new_entries)
29
30 if __name__ == "__main__":
31     while True:
32         add_sample_data()
33         time.sleep(10) # Ajoute des données toutes les 10 secondes
34
```

```
remplissage > dockerfile > ...
1 FROM python:3.9-slim
2
3 WORKDIR /app
4
5 COPY . .
6
7 CMD ["python", "script.py"]
8
```



2.SERVICE DE LECTURE

- Rôle : Lit et transforme les données générées par le service de remplissage.
- Technologies : Utilisation de json pour manipuler les fichiers et de bibliothèques comme time pour les mises à jour périodiques.
- Processus : Filtre ou enrichit les données avant de les rendre disponibles pour l'affichage.
- Interaction : Lit les données du volume shared-data et les met à jour pour l'affichage.

```
lecture > script.py > ...
1 import json
2 import os
3 import time
4
5 DATA_FILE = "/data/processed_table.json"
6
7 def read_data():
8     """Lit les données du fichier JSON et les affiche."""
9     if os.path.exists(DATA_FILE):
10         with open(DATA_FILE, 'r') as f:
11             data = json.load(f)
12             print("Données actuelles dans le fichier :", data)
13     else:
14         print("Aucun fichier trouvé ou données inexistantes.")
15
16 if __name__ == "__main__":
17     while True:
18         print("\nLecture des données...")
19         read_data()
20         time.sleep(10) # Lit les données toutes les 10 secondes
21
```

```
lecture > dockerfile > ...
1 FROM python:3.9-slim
2
3 WORKDIR /app
4
5 COPY . .
6
7 CMD ["python", "script.py"]
8
```



3. SERVICE D'AFFICHAGE

- Rôle : Fournit une interface web pour visualiser les données traitées.
- Technologies :
 1. Framework Flask pour une interface utilisateur simple.
 2. Templates HTML et CSS pour une visualisation claire (par exemple : tableau dynamique).
- Accès : Accessible via un navigateur web à un port configuré.
- Interaction : Lit les données transformées dans le volume partagé pour les afficher en temps réel.

```
affichage > app.py > ...
1 from flask import Flask, render_template, request, redirect, url_for
2 import json
3 import os
4
5 app = Flask(__name__)
6
7 DATA_FILE = "/data/processed_table.json"
8
9 def read_data():
10     if os.path.exists(DATA_FILE):
11         with open(DATA_FILE, 'r') as f:
12             return json.load(f)
13     return []
14
15 def write_data(new_entry):
16     data = read_data()
17     data.append(new_entry)
18     with open(DATA_FILE, 'w') as f:
19         json.dump(data, f, indent=4)
20
21 @app.route("/", methods=["GET"])
22 def home():
23     data = read_data()
24     return render_template("table.html", data=data)
25
26 @app.route("/add", methods=["POST"])
27 def add_entry():
28     nom = request.form["nom"]
29     age = request.form["age"]
30     ville = request.form["ville"]
31
32     new_entry = {"Nom": nom, "Âge": int(age), "Ville": ville}
33     write_data(new_entry)
34     return redirect(url_for("home"))
35
36 if __name__ == "__main__":
37     app.run(host="0.0.0.0", port=5000)
38
```

```
affichage > dockerfile > ...
1 FROM python:3.9-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install -r requirements.txt
7
8 COPY . .
9
10 CMD ["python", "app.py"]
11
```

```
affichage
├── templates
│   └── table.html
├── app.py
├── dockerfile
└── requirements.txt
```

Gestion des Étudiants Inscrits

Ajouter Un Nouveau Étudiant

Nom

Âge

Ville

Tableau des Données

Nom	Âge	Ville	Actions
BEN SAAD Abir	25	Kram Ouest	Supprimer
HABBOUBI Mariem	25	Tunis	Supprimer
BENALI Mariem	22	Rades	Supprimer
HAMROUNI Yassmine	21	Tunis	Supprimer
MEZLINI Bochra	29	Mateur	Supprimer

4. LOAD BALANCER

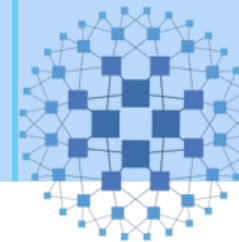
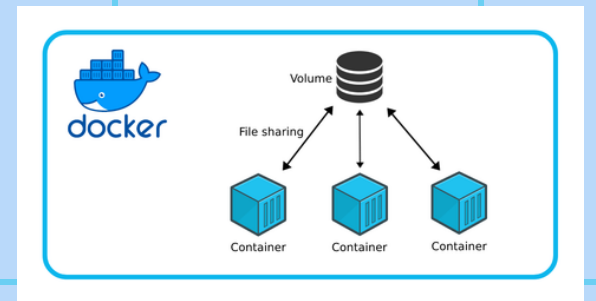
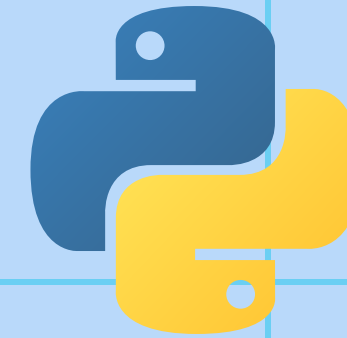
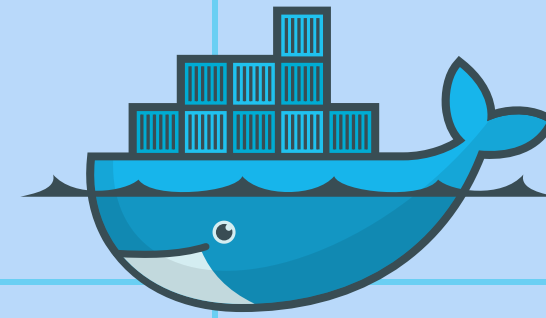
- Rôle : Assure une répartition optimale des requêtes entre plusieurs instances du service d'affichage.
- Technologies :
 1. HAProxy pour la répartition des charges.
 2. Configurations dynamiques pour ajuster les performances.
- Impact : Amélioration des performances globales et gestion des pics de trafic.
- Interaction : Route les requêtes vers les instances du service d'affichage en fonction des règles définies (round-robin).

```
load-balancer > ⚙️ haproxy.cfg
1 global
2   log stdout format raw local0
3
4 defaults
5   log global
6   mode http # Ajout du mode HTTP
7   option httplog
8   timeout connect 5000ms
9   timeout client 50000ms
10  timeout server 50000ms
11
12 frontend http_front
13   bind *:80
14   default_backend app_back
15
16 backend app_back
17   balance roundrobin
18   server app1 affichage-1:5000 check
19   server app2 affichage-1:5001 check
20
```

```
load-balancer > 🐳 dockerfile > ...
1 FROM haproxy:2.7
2
3 COPY haproxy.cfg /usr/local/etc/haproxy/haproxy.cfg
```



TECHNOLOGI ES UTILISEES



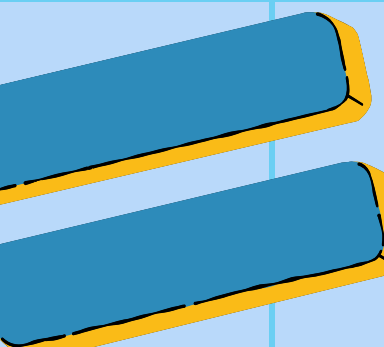
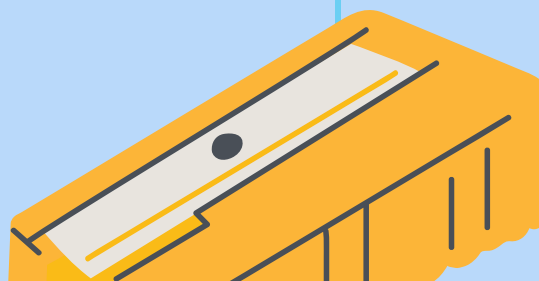
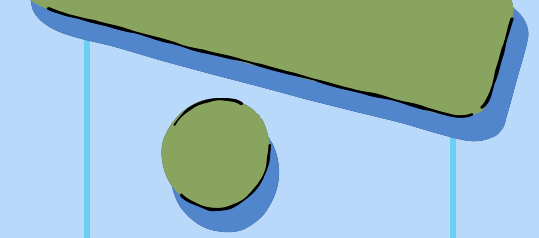
HAPROXY

{JSON}

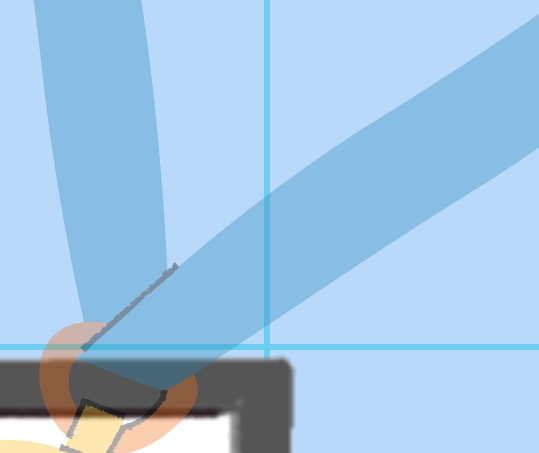
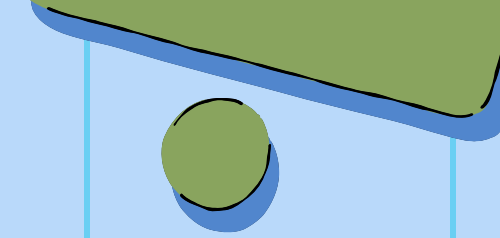


1. Docker : Conteneurisation et orchestration des services.
2. Python : Gestion des scripts de collecte, traitement et affichage.
3. Volumes Docker : Échange de données entre les services.
4. Flask : Interface web pour l'affichage des données.
5. HAProxy : Load balancing et optimisation des performances.
6. JSON : Format standard pour le stockage et l'échange de données entre services.

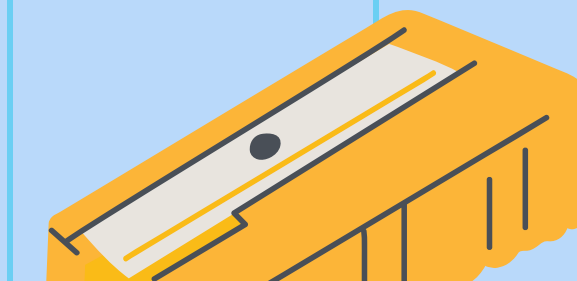
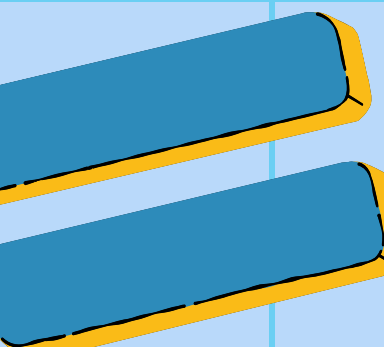


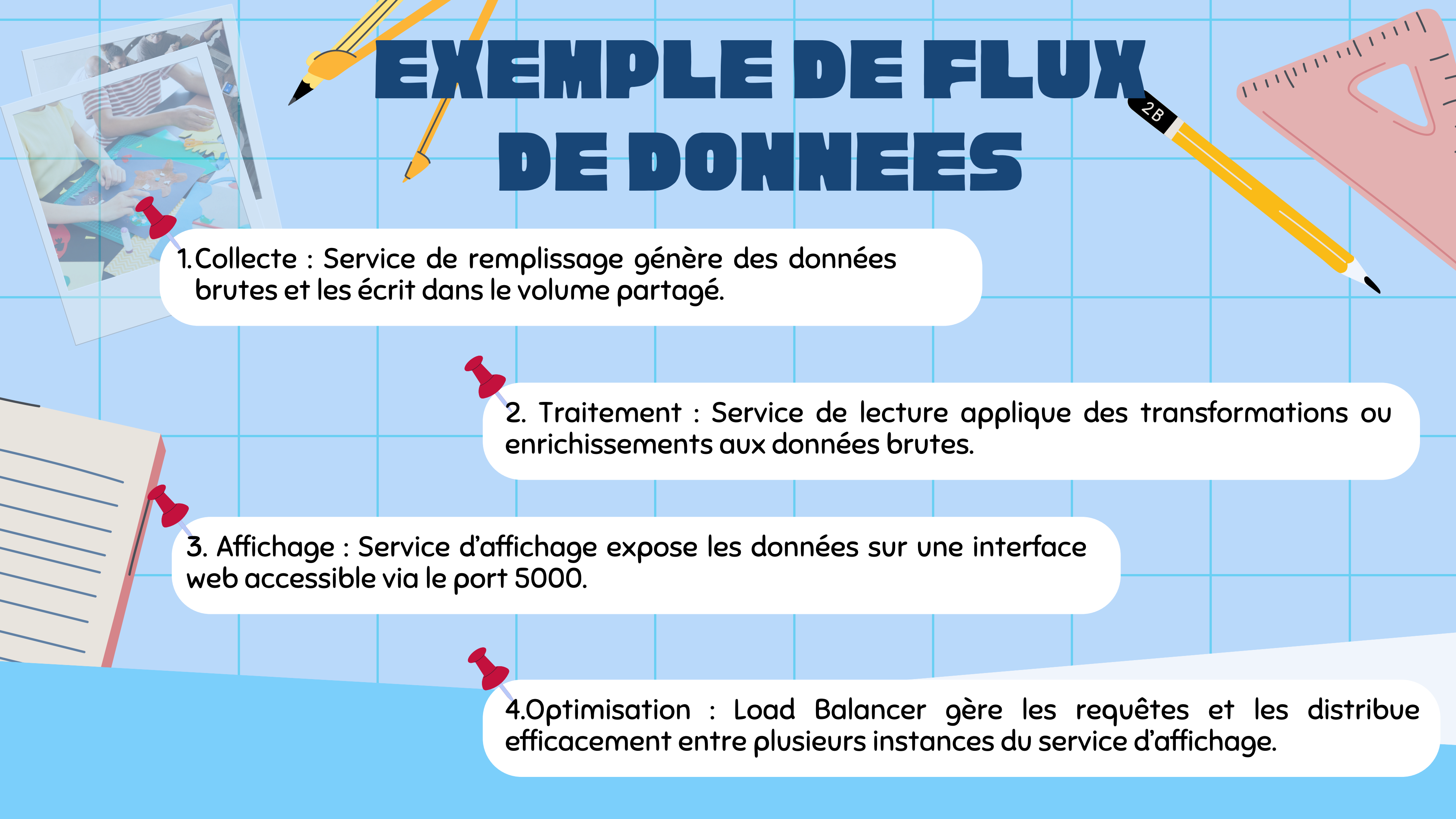


FONCTIONNALITES PRINCIPALES



- Traitement des Données : Centralisé dans des fichiers partagés (JSON).
- Interface Web : Tableau de bord intuitif pour l'analyse des données.
- Indépendance des Services : Chaque composant est autonome.
- Scalabilité : Load Balancer pour une réponse efficace aux pics de trafic.
- Interopérabilité : Volumes partagés pour faciliter les échanges de données entre les conteneurs.





EXEMPLE DE FLUX DE DONNEES



1. Collecte : Service de remplissage génère des données brutes et les écrit dans le volume partagé.



2. Traitement : Service de lecture applique des transformations ou enrichissements aux données brutes.



3. Affichage : Service d'affichage expose les données sur une interface web accessible via le port 5000.



4. Optimisation : Load Balancer gère les requêtes et les distribue efficacement entre plusieurs instances du service d'affichage.

DEPLOIEMENT ET EXECUTION



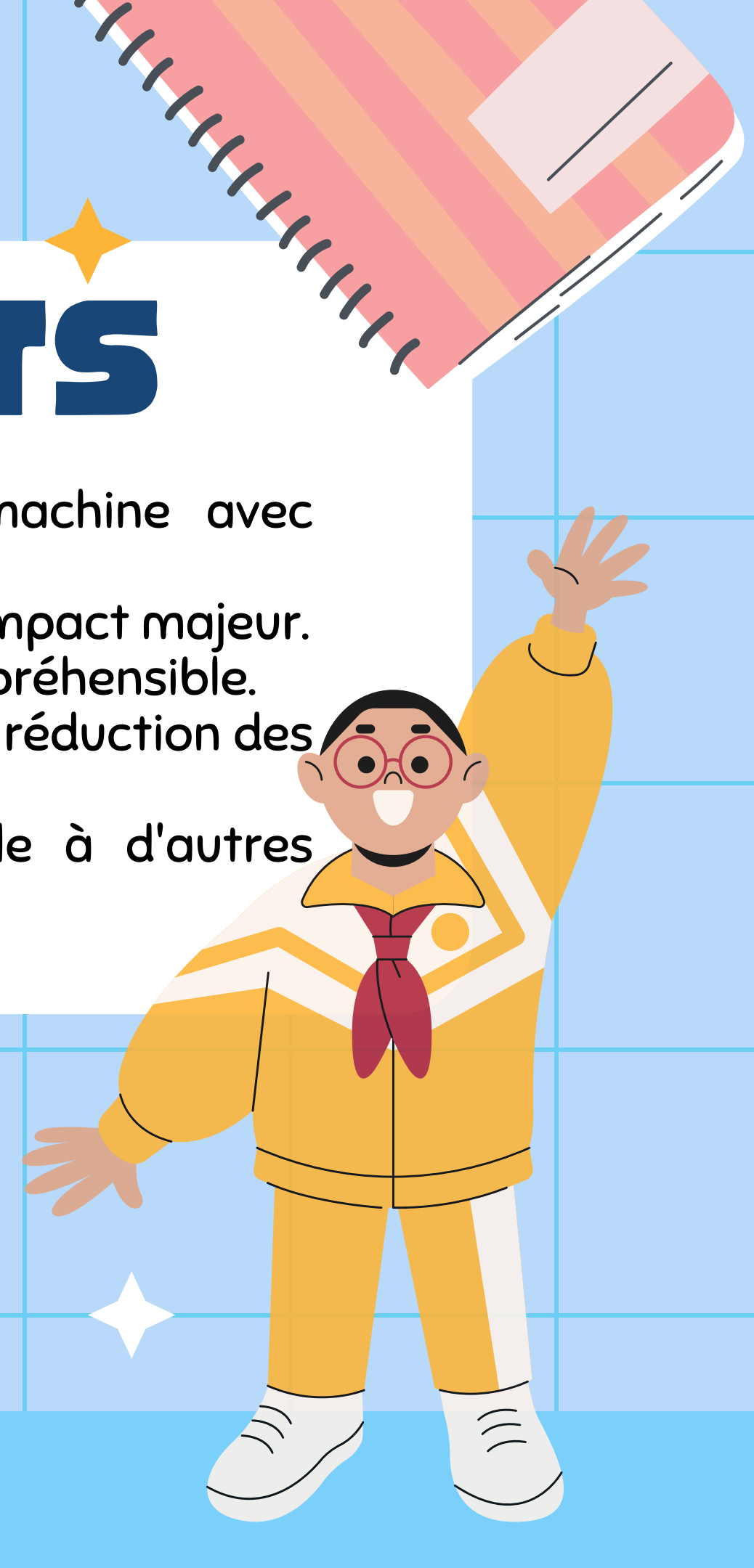


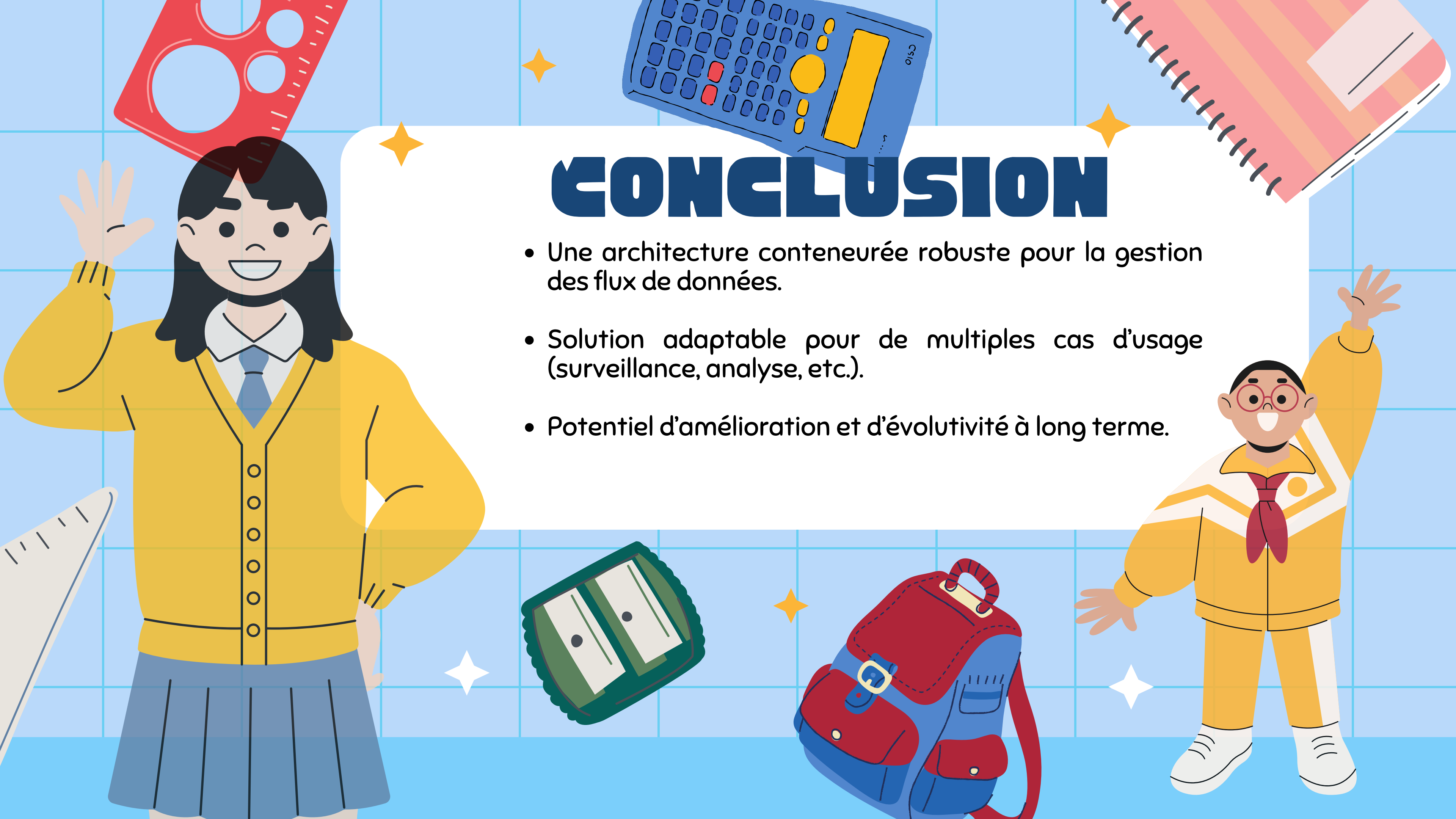
ETAPES :

1. Cloner le projet depuis le dépôt.
2. Lancer docker-compose up --build pour construire et démarrer les conteneurs.
3. Accéder aux services via les ports configurés :
 - Affichage : <http://localhost:5000>.
 - Load Balancer : <http://localhost:8080>.



POINTS FORTS

- Portabilité : Exécution sur n'importe quelle machine avec Docker.
 - Flexibilité : Ajout ou suppression de services sans impact majeur.
 - Simplicité : Configuration claire et facilement compréhensible.
 - Performances : Gestion optimisée des requêtes et réduction des temps de réponse.
 - Interopérabilité : Système facilement adaptable à d'autres sources ou types de données.
- 



CONCLUSION

- Une architecture conteneurée robuste pour la gestion des flux de données.
- Solution adaptable pour de multiples cas d'usage (surveillance, analyse, etc.).
- Potentiel d'amélioration et d'évolutivité à long terme.



MERCI POUR VOTRE ATTENTION!